RAG Assignment Report: Baseline vs. Re-Ranking

Name: Ishaan Giri **Roll no.:** 2025201091 **Repo:** github.com/Ishaan-Giri/lma-rag-reranking

1. Architecture Comparison

A bi-encoder reads the query and each chunk separately. Both go through the same transformer and are mean-pooled into one 384-dimensional vector each, and the only thing that gets compared is the cosine similarity between those two vectors. The query and the chunk never see each other. A cross-encoder works differently: it takes the query and the chunk together as one input (`[CLS] query [SEP] chunk`), so self-attention can look across the words of both texts at once, and it returns a single relevance score. This means a word in the query can attend directly to a specific word in the chunk, which isn't possible once the chunk has been squashed into one vector.

This also decides what can be done in advance. The bi-encoder's chunk vectors are computed once and stored in the index. The cross-encoder's score depends on the query, so nothing can be precomputed, and every (query, chunk) pair needs a full forward pass at the moment the question is asked.

A good example from the rulebook is the Space Combat chunk (132). Most of it is about the combat steps, and the line "if the defender announces a retreat, the attacker cannot announce a retreat" is just one bullet inside it. I think the single vector for that chunk is mostly about combat in general, so this bullet gets watered down. The baseline put the chunk at #7 for a question about the attacker retreating, while the cross-encoder could match "attacker" and "retreat" to that bullet and put it at #1.

2. Qualitative & Rank Shift Analysis

2.1 Rank changes for the failure queries

Out of 77 stress-test queries, re-ranking moved the right chunk from rank 4–9 into the top 3 in 10 cases.

Category	**Query (short)**	**Right chunk**	**Rank change**
Keyword trap	Can we take a trade back once it's done?	170 (Transactions)	#6 → #1
Keyword trap	Can I give commodities away for free?	170	#6 → #1
Keyword trap	Can I swap action cards with my neighbor?	170	#7 → #1
Keyword trap	Is there a limit on trading with one neighbor?	170	#6 → #1
Exception	Can I trade in the middle of a space combat?	170	#9 → #1
Exception	Any case where I can't score an objective I qualify for?	91 (Objectives)	#5 → #3
Negation	Can the attacker announce a retreat after the defender did?	132 (Space Combat)	#7 → #1
Negation	Can Sustain Damage save a ship from direct destruction?	153 (Sustain Damage)	#4 → #1
Buried	Do the Creuss wormholes stay if that player is eliminated?	58 (Elimination)	#4 → #1
Buried	Can I use a captured planet's resources right away?	53 (Control)	#6 → #2
Counter-example	Can I spend trade goods for extra votes?	23 / 79 / 161	#1 → #2

Overall, across all 77 queries: 56 were already correct in the baseline, **10 were fixed**, 5 were missed by both pipelines (in the top 10 but outside the top 3), and 6 were not in the top 10 at all, so re-ranking couldn't help them.

Why the baseline failed. In the keyword-trap queries the baseline picked chunks about the main noun in the question. "Commodities" brought up three Commodities chunks and "action cards" brought up two Action Cards chunks. They use the same words as the question but don't answer it. The answer is in the Transactions rule (chunk 170), which talks about the action itself: exchanging, giving and undoing.

Why chunk 170 shows up five times. All the trading rules sit in that one chunk, and for every "trade" question the chunks about the topic words beat it. It also says something about how I built the stress set: several of my questions point at the same rule, so these five fixes shouldn't be counted as five separate pieces of evidence.

Limits of a re-ranker. It can only reorder the 10 chunks it is given. For KW02 ("when does the agenda phase start") the right chunk was at bi-encoder rank #41, so there was nothing for the re-ranker to promote. A bigger top-k or a hybrid BM25 search would be the fix there. The re-ranker isn't perfect either. It never pushed a baseline hit out of the top 3, but for some queries it moved the right chunk further down (for example KW07, #4 → #8). My guess is that this is because the MS MARCO model was trained on web search and not on rulebooks.

2.2 Answer 1 (baseline top-3) vs. Answer 2 (re-ranked top-3)

Same prompt, same model, temperature 0. The prompt says "answer only from the excerpts; otherwise say they don't cover it."

Query	**Answer 1**	**Answer 2**	**Gold chunk in context (A1 / A2)**
Take a trade back	Refused ("not covered")	Correct: cannot be undone	No / Yes
Give commodities free	Partial answer, then refused	Correct	No / Yes
Swap action cards	Refused	Correct	No / Yes
Trading limit	Refused	Correct	No / Yes
Trade during combat	Refused	Correct	No / Yes
Objective can't be scored	**Sounded complete but missed the home-system rule**	Correct, includes the home-system rule	No / Yes
Attacker retreat	Refused	Correct	No / Yes
Sustain Damage vs. destroy	Hedged, no clear answer	Correct: No	No / Yes
Creuss wormholes	Refused	Correct	No / Yes
Captured planet resources	Refused	Correct (needs two chunks)	No / Yes
Trade goods for votes (counter-example)	Correct (mentions Bribery card)	Correct (mentions Bribery card)	Yes / Yes

Hallucination, omission or misstatement? I didn't see any outright hallucination in either pipeline. The baseline mostly failed by leaving things out. For the objective query it listed "already scored" and "one per status phase" but missed the home-system rule. For the commodities query it first explained the wrong rule (commodities turning into trade goods) and then gave up, and for Sustain Damage it only hedged.

Why refusals? 7 of the 10 baseline answers were refusals instead of wrong answers, because the prompt tells the model to answer only from the excerpts. I didn't test it, but without that line

Gemini would probably have answered from what it already knows about Twilight Imperium, and we would never have noticed that retrieval had failed.

Most dangerous failure. For me it is the objective query. The baseline answer reads fluently and looks complete, and nothing in it hints that a rule is missing, so a player would wrongly think they can score the objective. A refusal at least tells you something went wrong.

What the counter-example showed. For the trade-goods query the re-ranker moved the Bribery errata to #1 and the right chunk (23) fell out of the top 3, but both answers were still correct because the same rule is written in three places (chunks 23, 79 and 161). So the rulebook's own repetition saved the answer, and a change in rank doesn't always change the final answer.

3. Latency Profiling

Measured on CPU (6 threads) after warm-up: 77 queries × 20 repetitions = 1540 samples per retrieval/re-rank stage. The LLM time comes from 22 real API calls.

Stage	Mean (ms)	Std (ms)	p95 (ms)
Retrieval: embed query + top-10 search	8.3	2.3	9.7
Re-rank: cross-encoder on 10 pairs	214.5	21.9	250.5
LLM generation (Gemini)	2819	1459	5672

Pipeline	Retrieval stage only	End-to-end (with LLM)
1: Baseline	8.3 ms	2828 ms
2: Re-ranking	222.8 ms	3042 ms

Retrieval is about 27× slower with re-ranking (8.3 → 222.8 ms), but the full pipeline is only about 8% slower (2828 → 3042 ms). The reason is that the LLM call takes about 2.8 s in both pipelines, so an extra 0.2 s hardly shows, and it is smaller than the normal swing in the LLM time itself (std 1.5 s). For this project I think the cost is worth it: for roughly 0.2 s, 10 of 77 queries went from a refusal or an incomplete answer to a correct one. On a GPU the cross-encoder should be a lot faster (I didn't measure this), so the cost would matter even less. In a real-time chatbot 200 ms is probably fine, but it could be noticeable if what matters is how quickly the first streamed word appears.

4. Scaling Question

Measured: cross-encoder on the top 10 ≈ 200 ms; on all 225 chunks ≈ 4.5 s (≈ 20 ms per pair); bi-encoder search ≈ 8 ms.

Collection size	Cross-encoder on everything (linear extrapolation)
225 chunks (this document)	4.5 s per query (measured)
10,000 chunks	≈ 201 s per query
1,000,000 chunks	≈ 20,100 s ≈ 5.6 hours per query

A cross-encoder has to run a full transformer pass for every (query, chunk) pair, so for each query its cost grows linearly with the number of chunks. This work can't be cached or indexed beforehand either, because the input includes the query, which we don't know until the user asks; there is no per-chunk representation that is independent of the query. The bi-encoder is different: chunks are embedded once offline (225 chunks took about 3.4 s), and each query only needs one encoder pass and a fast vector search (about 8 ms here).

The two-stage design lets each model do what it is good at. The bi-encoder gives recall: it cheaply shortlists 10 candidates out of everything. The cross-encoder gives precision: it carefully orders just

those 10, at a cost (about 200 ms) that doesn't depend on how big the collection is. The weakness is that the re-ranker can only work with what the first stage found. In 6 of the 77 queries the right chunk wasn't in the top 10 at all (for example rank #41 for KW02), so the cross-encoder couldn't help. A larger top-k or a hybrid search (BM25 + dense) would help, but a larger top-k means more cross-encoder time.